

Assignment 2 MA-347

Arpa Banik

2024-02-10

Question 1

```
# Load necessary libraries
library(tidyverse)
```

```
## — Attaching core tidyverse packages — tidyverse 2.0.0 —
## ✓ dplyr      1.1.4      ✓ readr      2.1.5
## ✓ forcats    1.0.0      ✓ stringr   1.5.1
## ✓ ggplot2    3.4.4      ✓ tibble    3.2.1
## ✓ lubridate  1.9.3      ✓ tidyr     1.3.1
## ✓ purrr      1.0.2
## — Conflicts — tidyverse_conflicts() —
## ✖ dplyr::filter() masks stats::filter()
## ✖ dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(caret)
```

```
## Loading required package: lattice
##
## Attaching package: 'caret'
##
## The following object is masked from 'package:purrr':
##
## lift
```

```
library(pROC)
```

```
## Type 'citation("pROC")' for a citation.
##
## Attaching package: 'pROC'
##
## The following objects are masked from 'package:stats':
##
## cov, smooth, var
```

```
library(gains)
tinytex::tlmgr_install()
library(tinytex)
# Import the dataset
data <- read.csv("F:/Arpa/attachments/PredResults.csv")

# (a) Function to calculate confusion matrix for a given cutoff
cutoff <- c(0.25, 0.5, 0.75)
calculate_confusion_matrix <- function(cutoff) {
  predicted_labels <- ifelse(data$Probabilities >= cutoff, 1, 0)
  confusionMatrix(table( predicted_labels,data$Actual))
}
# Produce confusion matrix for each cutoff
confusion_matrices <- lapply(cutoff, calculate_confusion_matrix)
names(confusion_matrices) <- paste("Cutoff", cutoff)
print(confusion_matrices)
```

```
## `$Cutoff 0.25`
## Confusion Matrix and Statistics
##
##
## predicted_labels 0 1
##                0 9 0
##                1 8 3
##
##                Accuracy : 0.6
##                95% CI : (0.3605, 0.8088)
##                No Information Rate : 0.85
##                P-Value [Acc > NIR] : 0.99867
```

```

##
##          Kappa : 0.2523
##
## Mcnemar's Test P-Value : 0.01333
##
##          Sensitivity : 0.5294
##          Specificity : 1.0000
##          Pos Pred Value : 1.0000
##          Neg Pred Value : 0.2727
##          Prevalence : 0.8500
##          Detection Rate : 0.4500
##          Detection Prevalence : 0.4500
##          Balanced Accuracy : 0.7647
##
##          'Positive' Class : 0
##
##
## $`Cutoff 0.5`
## Confusion Matrix and Statistics
##
## predicted_labels  0  1
##                0 15  0
##                1  2  3
##
##          Accuracy : 0.9
##          95% CI : (0.683, 0.9877)
##          No Information Rate : 0.85
##          P-Value [Acc > NIR] : 0.4049
##
##          Kappa : 0.6923
##
## Mcnemar's Test P-Value : 0.4795
##
##          Sensitivity : 0.8824
##          Specificity : 1.0000
##          Pos Pred Value : 1.0000
##          Neg Pred Value : 0.6000
##          Prevalence : 0.8500
##          Detection Rate : 0.7500
##          Detection Prevalence : 0.7500
##          Balanced Accuracy : 0.9412
##
##          'Positive' Class : 0
##
##
## $`Cutoff 0.75`
## Confusion Matrix and Statistics
##
## predicted_labels  0  1
##                0 17  1
##                1  0  2
##
##          Accuracy : 0.95
##          95% CI : (0.7513, 0.9987)
##          No Information Rate : 0.85
##          P-Value [Acc > NIR] : 0.1756
##
##          Kappa : 0.7727
##
## Mcnemar's Test P-Value : 1.0000
##
##          Sensitivity : 1.0000
##          Specificity : 0.6667
##          Pos Pred Value : 0.9444
##          Neg Pred Value : 1.0000
##          Prevalence : 0.8500
##          Detection Rate : 0.8500
##          Detection Prevalence : 0.9000
##          Balanced Accuracy : 0.8333
##
##          'Positive' Class : 0
##
##

```

b. Function to calculate error rate, sensitivity, and specificity for a given confusion matrix

```
# Convert Actual column to factor with levels
data$Actual <- factor(data$Actual)
confusion_matrices <- lapply(cutoff, function(cutoff) {
  predicted_labels <- factor(ifelse(data$Probabilities >= cutoff, 1, 0), levels = levels(data$Actual))
  confusionMatrix(predicted_labels, data$Actual)
})
# Print confusion matrices and calculate error rate, sensitivity, specificity
for (i in seq_along(cutoff)) {
  cat(paste("$`Cutoff", cutoff[i], "`\n"))
  error_rate <- 1 - confusion_matrices[[i]]$overall["Accuracy"]
  sensitivity <- confusion_matrices[[i]]$byClass["Sensitivity"]
  specificity <- confusion_matrices[[i]]$byClass["Specificity"]
  cat("\nError Rate:", error_rate, "\n")
  cat("Sensitivity:", sensitivity, "\n")
  cat("Specificity:", specificity, "\n\n")
}
```

```
## $`Cutoff 0.25 `
##
## Error Rate: 0.4
## Sensitivity: 0.5294118
## Specificity: 1
##
## $`Cutoff 0.5 `
##
## Error Rate: 0.1
## Sensitivity: 0.8823529
## Specificity: 1
##
## $`Cutoff 0.75 `
##
## Error Rate: 0.05
## Sensitivity: 1
## Specificity: 0.6666667
```

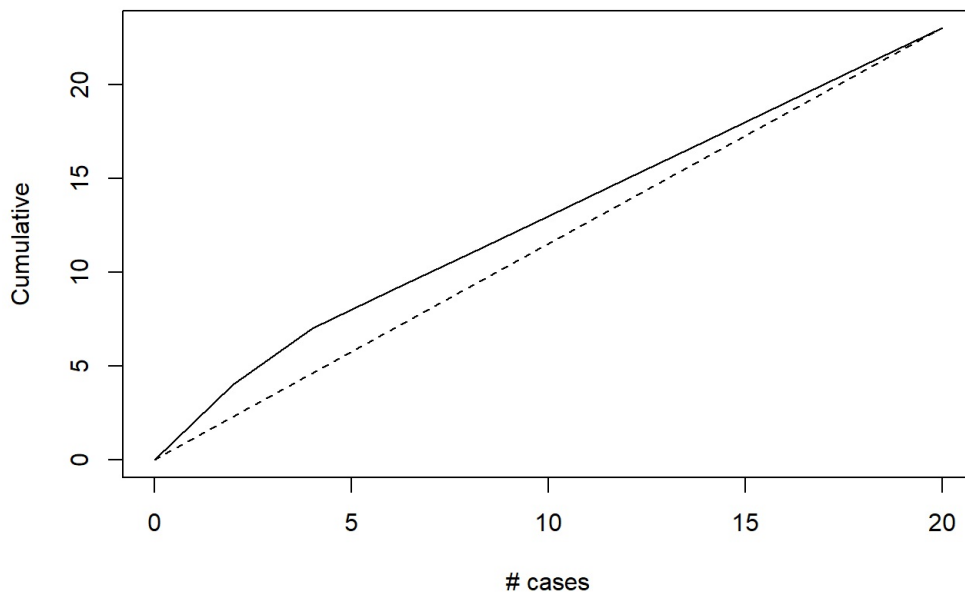
(c) The best cutoff value is at 0.75 since the error rate 0.05 is the smallest at that point.

d. Lift Chart

```
data$Actual <- as.numeric(data$Actual)
gain <- gains(data$Actual, data$Probabilities, groups=10)
gain
```

##	Depth								
##	of		Cume	Mean	Cume	Cume Pct		Cume	Mean
##	File	N	N	Resp	Mean	of Total	Lift	Lift	Model
##					Resp	Resp	Index		Score
##	10	2	2	2.00	2.00	17.4%	174	174	0.80
##	20	2	4	1.50	1.75	30.4%	130	152	0.57
##	30	2	6	1.00	1.50	39.1%	87	130	0.47
##	40	2	8	1.00	1.38	47.8%	87	120	0.42
##	50	2	10	1.00	1.30	56.5%	87	113	0.35
##	60	2	12	1.00	1.25	65.2%	87	109	0.25
##	70	2	14	1.00	1.21	73.9%	87	106	0.11
##	80	2	16	1.00	1.19	82.6%	87	103	0.08
##	90	2	18	1.00	1.17	91.3%	87	101	0.04
##	100	2	20	1.00	1.15	100.0%	87	100	0.01

```
plot(c(0, gain$cume.pct.of.total * sum(data$Actual)) ~ c(0, gain$cume.obs),
     xlab = '# cases', ylab = 'Cumulative', main = '', type = 'l')
lines(c(0, sum(data$Actual)) ~ c(0, dim(data)[1]), lty = 2)
```



Question 2

```
# Load necessary libraries
library(dplyr)
library(tidyr)
library(caTools)
library(glmnet)
```

```
## Loading required package: Matrix
```

```
##
## Attaching package: 'Matrix'
```

```
## The following objects are masked from 'package:tidyr':
##
## expand, pack, unpack
```

```
## Loaded glmnet 4.1-8
```

```
library(ROCR)
# Load the data
eBay_df <- read.csv("F:/Arpa/attachments/eBayAuctions.csv")

# (a) Randomly split the dataset into training and validation sets
# partition data
set.seed(347)
train_indices <- sample(1:nrow(eBay_df), 0.6 * nrow(eBay_df))
train_set <- eBay_df[train_indices, ]
validation_set <- eBay_df[-train_indices, ]
```

b. Use the first four variables in the table above as predictors to fit a logistic regression model for this classification problem.

```
logistic_model <- glm(Competitive ~ sellerRating + Duration + ClosePrice + currency,
                      data = train_set,
                      family = "binomial")
summary(logistic_model)
```

```
##
## Call:
## glm(formula = Competitive ~ sellerRating + Duration + ClosePrice +
##       currency, family = "binomial", data = train_set)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)  -1.400e-02  3.072e-01  -0.046  0.96366
## sellerRating -9.158e-06  1.078e-05  -0.849  0.39567
## Duration     -5.428e-03  3.406e-02  -0.159  0.87337
## ClosePrice    1.083e-02  1.740e-03   6.224 4.83e-10 ***
## currencyGBP   7.908e-01  2.584e-01   3.061 0.00221 **
## currencyUS   -9.747e-02  1.613e-01  -0.604  0.54575
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 1625.5  on 1182  degrees of freedom
## Residual deviance: 1540.7  on 1177  degrees of freedom
## AIC: 1552.7
##
## Number of Fisher Scoring iterations: 5
```

```
validation_set$predicted_probabilities <- predict(logistic_model, newdata = validation_set, type = "response")
# Display the predicted probabilities
head(validation_set$predicted_probabilities)
```

```
## [1] 0.4580341 0.4580341 0.4580341 0.4580341 0.4580341 0.4580341
```

c. Write down the fitted logistic regression model

$$\text{Logit}(P) = -0.014 + 0.791 * (\text{currencyGBP}) - 0.097 * (\text{currencyUS}) - 9.16e-06 * (\text{sellerRating}) - 0.0054 * (\text{Duration}) + 0.011 * (\text{ClosePrice})$$

d. Describe all of the dummy variables used in the logistic regression model.

In the logistic regression model, two dummy variables were created for the `currency` predictor: `currencyGBP` and `currencyUS`. These dummy variables encode the categorical variable `currency`, representing whether the transaction was in British Pounds (GBP) or US Dollars (USD).

e. Interpret the estimated logistic regression coefficient associated with the predictors `Duration` and `currency`.

```
#interpret duration
duration <- exp(-0.0054)
100*(1-duration)
```

```
## [1] 0.5385446
```

```
#interpret currency = US
US <- exp(-0.097)
100*(1-US)
```

```
## [1] 9.244399
```

```
#interpret currency = GBP
GBP <- exp(-0.791)
100*(1-GBP)
```

```
## [1] 54.66088
```

Duration While holding all other variables constant, for a one unit increase in the duration of the auction, the odds of the auction being competitive increase by a multiplicative factor of 0.995; the change increases the odds of a competitive auction by 0.538%

currencyGBP While holding all other variables constant, for a one US dollar increase in the currency of the auction, the odds of the auction being competitive increase by a multiplicative factor of 0.907; the change increases the odds of a competitive auction by 9.244%.

currencyUS While holding all other variables constant, for a one British pound increase in the currency of the auction, the odds of the auction being competitive increase by a multiplicative factor of 0.453; the change increases the odds of a competitive auction by 54.661%.

f. Produce predicted probabilities of competitive auctions for transactions in your validation set.

```
# Predict probabilities on the validation set
validation_probs <- predict(logistic_model, newdata = validation_set, type = "response")
```

g. Produce ROCs on both the training and validation data

```
# Create ROC curve on training data
train_preds <- predict(logistic_model, newdata = train_set, type = "response")
train_roc <- roc(train_set$Competitive, train_preds)
```

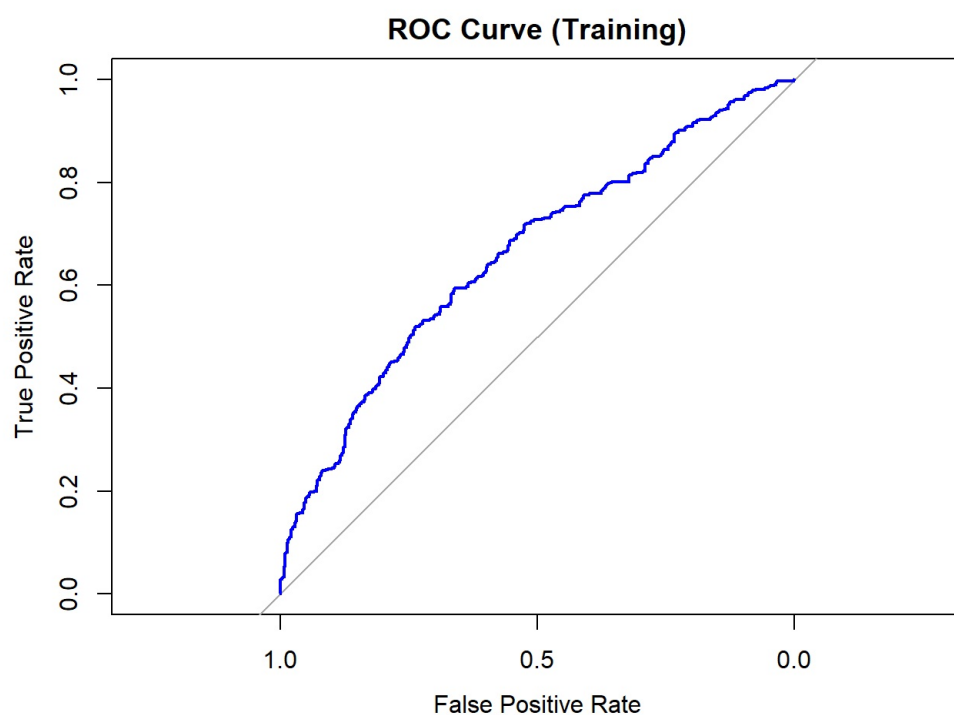
```
## Setting levels: control = 0, case = 1
```

```
## Setting direction: controls < cases
```

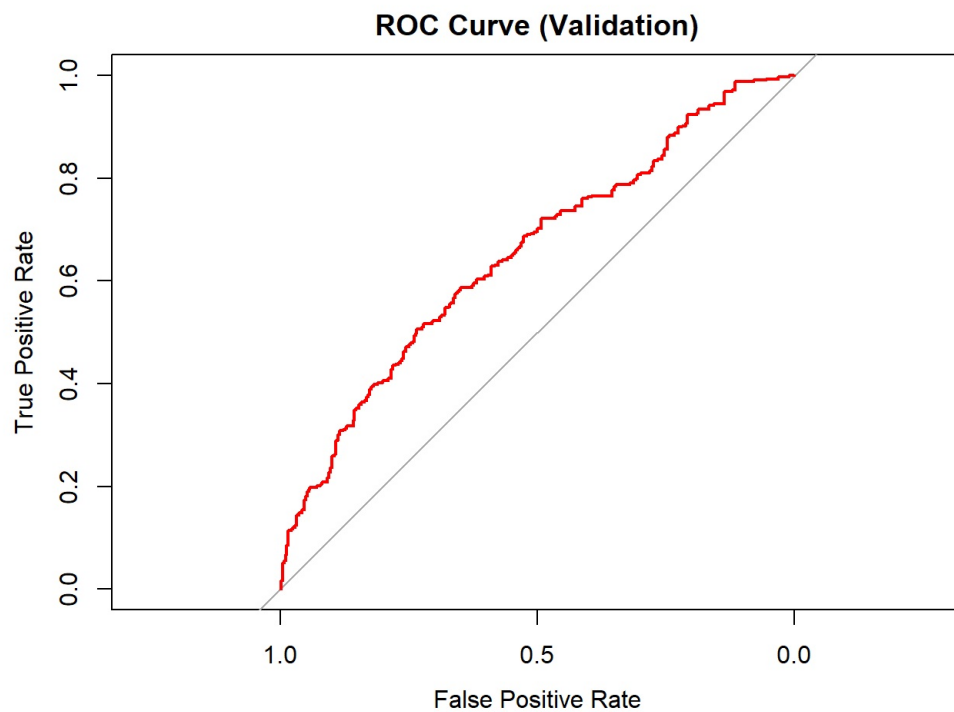
```
# Create ROC curve on validation data
validation_roc <- roc(validation_set$Competitive, validation_probs)
```

```
## Setting levels: control = 0, case = 1
## Setting direction: controls < cases
```

```
# Plot ROC curve for training data
plot(train_roc, col = "blue", main = "ROC Curve (Training)", xlab = "False Positive Rate", ylab = "True Positive Rate")
```



```
# Plot ROC curve for validation data
plot(validation_roc, col = "red", main = "ROC Curve (Validation)", xlab = "False Positive Rate", ylab = "True Positive Rate")
```



h. Report the AUCs on both the training and validation data

```
# AUC for training data
train_auc <- auc(train_roc)
cat("Training AUC: ", train_auc, "\n")
```

```
## Training AUC: 0.6610356
```

```
# AUC for validation data
validation_auc <- auc(validation_roc)
cat("Validation AUC: ", validation_auc, "\n")
```

```
## Validation AUC: 0.6513222
```

Question 3

```
# Given information
true_positive <- 30
true_negative <- 920
false_positive <- 88 - true_positive
false_negative <- 952 - true_negative

# Construct the confusion matrix
confusion_matrix <- matrix(c(true_negative, false_positive, false_negative, true_positive), nrow = 2, byrow = TRUE)
colnames(confusion_matrix) <- c("Predicted Non-Fraudulent", "Predicted Fraudulent")
rownames(confusion_matrix) <- c("Actual Non-Fraudulent", "Actual Fraudulent")

# Print the confusion matrix
confusion_matrix
```

```
##
##          Predicted Non-Fraudulent Predicted Fraudulent
## Actual Non-Fraudulent           920                58
## Actual Fraudulent              32                 30
```

```
# Calculate overall error rate
overall_error_rate <- (false_positive + false_negative) / sum(confusion_matrix)
overall_error_rate
```

```
## [1] 0.08653846
```